Name	Vedant Dinkar Patil
UID no.	2024300170
Experiment No.	8

AIM:	To understand the concept of Fuzzy logic
PROBLEM STATEMENT :	To implement Fuzzy Sets and Fuzzy Relations for a given problem statement
Theory:	**Fuzzy Logic** Fuzzy Logic is a method used to make decisions when information is not completely true or false. In normal or **Boolean logic**, a statement is either **0 (False)** or **1 (True)**. But in real life, many situations are not simply true or false. For example, we may say that the temperature is **slightly hot**, **very hot**, or **comfortable**. Fuzzy logic is useful for handling such situations. Fuzzy logic was introduced by **Lotfi A. Zadeh in 1965**. It works with **degrees of truth**, usually between 0 and 1. For example, if the temperature is 30°C, we can say that it is hot with a membership value of 0.7 and comfortable with a value of 0.3. This makes fuzzy logic more similar to the way humans think and make decisions. The main concept of fuzzy logic is the **fuzzy set**. A fuzzy set contains elements that can belong to the set by different degrees. The degree of belonging is represented by a **membership function**, whose value ranges from 0 to 1. A value of 0 means the element does not belong to the set, while 1 means it completely belongs to the set. A fuzzy logic system generally has four main parts: **fuzzification, rule base, inference engine, and defuzzification**. In fuzzification, normal input values are converted into fuzzy values. The rule base contains simple **IF-THEN rules**, such as "IF temperature is high THEN fan speed is fast." The inference engine applies these rules to the given input. Finally, defuzzification converts the fuzzy result back into a normal numerical value that can be used by a machine or system. Fuzzy logic is commonly used in **washing machines, air conditioners, cameras, automobiles, robotics, and medical systems**. For example, a washing machine can decide the washing time and water level based on how dirty the clothes are instead of using only fixed values. The main advantage of fuzzy logic is that it can handle **uncertain, approximate, and imprecise information** and does not always require a complex mathematical model. It is

	also relatively easy to understand because its rules are similar to human reasoning. However, designing suitable membership functions and rules can sometimes be difficult. In simple words, **fuzzy logic allows a computer or machine to make decisions in a more human-like way by working with partial truth instead of only yes or no decision.**

PROGRAM:	#include <iostream>
	#include <vector>
	#include <iomanip>
	#include <sstream>
	#include <string>
	#include <algorithm>
	#include <cmath>
	#include <cctype>
	
	using namespace std;
	string formatNumber(double value)
	{
	  if (fabs(value) < 1e-9)
	  value = 0.0;
	
	  ostringstream out;
	  out << fixed << setprecision(2) << value;
	
	  string s = out.str();
	  while (!s.empty() && s.back() == '0')
	  s.pop_back();
	
	  if (!s.empty() && s.back() == '.')
	  s.pop_back();
	
	  return s;
	}
	
	void printFuzzySet(const vector<double>& A,
	  const vector<int>& elements)
	{
	  cout << "{ ";

```
    for (size_t i = 0; i < A.size(); i++)
    {
        cout << formatNumber(A[i]) << "/" << elements[i];

        if (i != A.size() - 1)
            cout << " + ";
    }

    cout << " }";
}

vector<double> fuzzyUnion(const vector<double>& A,
                          const vector<double>& B)
{
    vector<double> result(A.size());

    for (size_t i = 0; i < A.size(); i++)
        result[i] = max(A[i], B[i]);

    return result;
}
vector<double> fuzzyIntersection(const vector<double>& A,
                                 const vector<double>& B)
{
    vector<double> result(A.size());

    for (size_t i = 0; i < A.size(); i++)
        result[i] = min(A[i], B[i]);

    return result;
}

vector<double> fuzzyComplement(const vector<double>& A)
{
    vector<double> result(A.size());

    for (size_t i = 0; i < A.size(); i++)
        result[i] = 1.0 - A[i];
```

```
    return result;
}

void printResult(const string& name,
                 const vector<double>& result,
                 const vector<int>& elements)
{
    cout << name << " = ";
    printFuzzySet(result, elements);
    cout << endl;
}

// QUESTION 7
void question7()
{
    cout <<
"\n=============================================================\n";
    cout << "QUESTION 7 - FUZZY SET OPERATIONS\n";
    cout <<
"=============================================================\n";

    vector<int> elements = {0, 1, 2, 3, 4};

    vector<double> A = {
        0.1, 0.2, 0.4, 0.6, 1.0
    };

    vector<double> B = {
        1.0, 0.5, 0.7, 0.3, 0.0
    };

    cout << "\nGiven:\n";

    cout << "A = ";
    printFuzzySet(A, elements);
    cout << "\n";

    cout << "B = ";
    printFuzzySet(B, elements);
    cout << "\n\n";
```

```
// Basic operations
vector<double> A_union_B = fuzzyUnion(A, B);
vector<double> A_inter_B = fuzzyIntersection(A, B);

vector<double> A_bar = fuzzyComplement(A);
vector<double> B_bar = fuzzyComplement(B);

printResult("(a) A U B",
        A_union_B,
        elements);
printResult("(b) A ∩ B",
        A_inter_B,
        elements);
printResult("(c) A-bar",
        A_bar,
        elements);
printResult("(d) B-bar",
        B_bar,
        elements);
printResult("(e) A U A-bar",
        fuzzyUnion(A, A_bar),
        elements);
printResult("(f) A ∩ A-bar",
        fuzzyIntersection(A, A_bar),
        elements);
printResult("(g) B U B-bar",
        fuzzyUnion(B, B_bar),
        elements);
printResult("(h) B ∩ B-bar",
        fuzzyIntersection(B, B_bar),
        elements);
printResult("(i) A ∩ B-bar",
        fuzzyIntersection(A, B_bar),
        elements);
printResult("(j) A U B-bar",
        fuzzyUnion(A, B_bar),
        elements);
printResult("(k) B ∩ A-bar",
        fuzzyIntersection(B, A_bar),
```

```
            elements);
    printResult("(l) B U A-bar",
            fuzzyUnion(B, A_bar),
            elements);
    vector<double> m_result =
        fuzzyComplement(A_union_B);

    printResult("(m) (A U B)-bar",
            m_result,
            elements);
    vector<double> n_result =
        fuzzyIntersection(A_bar, B_bar);

    printResult("(n) A-bar ∩ B-bar",
            n_result,
            elements);
}

// QUESTION 9
void question9()
{
    cout <<
"\n================================================================\n";
    cout << "QUESTION 9 - ALGEBRAIC AND BOUNDED OPERATIONS\n";
    cout <<
"================================================================\n";

    vector<int> elements = {1, 2, 3, 4};

    vector<double> A = {
        0.2, 0.3, 0.4, 0.5
    };

    vector<double> B = {
        0.1, 0.2, 0.2, 1.0
    };

    cout << "\nGiven:\n";

    cout << "A = ";
```

```
printFuzzySet(A, elements);
cout << "\n";

cout << "B = ";
printFuzzySet(B, elements);
cout << "\n\n";

vector<double> algebraicSum(A.size());
vector<double> algebraicProduct(A.size());
vector<double> boundedSum(A.size());
vector<double> boundedDifference(A.size());

for (size_t i = 0; i < A.size(); i++)
{
   algebraicSum[i] =
      A[i] + B[i] - A[i] * B[i];

   algebraicProduct[i] =
      A[i] * B[i];

   boundedSum[i] =
      min(1.0, A[i] + B[i]);

   boundedDifference[i] =
      max(0.0, A[i] - B[i]);
}

cout << "(a) Algebraic Sum\n";
cout << "    Formula: A + B - A*B\n";
printResult("    Result",
          algebraicSum,
          elements);

cout << "\n(b) Algebraic Product\n";
cout << "    Formula: A*B\n";
printResult("    Result",
          algebraicProduct,
          elements);

cout << "\n(c) Bounded Sum\n";
```

```
    cout << "    Formula: min(1, A+B)\n";
    printResult("    Result",
                boundedSum,
                elements);

    cout << "\n(d) Bounded Difference\n";
    cout << "    Formula: max(0, A-B)\n";
    printResult("    Result",
                boundedDifference,
                elements);
}

// QUESTION 2
void question2()
{
    cout <<
"\n============================================================\n";
    cout << "QUESTION 2 - FUZZY CARTESIAN PRODUCT\n";
    cout <<
"============================================================\n";

    vector<string> X = {"x1", "x2", "x3"};
    vector<string> Y = {"y1", "y2"};

    vector<double> A = {
        0.3, 0.7, 1.0
    };

    vector<double> B = {
        0.4, 0.9
    };

    cout << "\nGiven:\n";

    cout << "A = { ";
    for (size_t i = 0; i < A.size(); i++)
    {
        cout << formatNumber(A[i]) << "/" << X[i];

        if (i != A.size() - 1)
```

```
        cout << " + ";
    }
    cout << " }\n";

    cout << "B = { ";
    for (size_t i = 0; i < B.size(); i++)
    {
       cout << formatNumber(B[i]) << "/" << Y[i];

       if (i != B.size() - 1)
          cout << " + ";
    }
    cout << " }\n\n";

    cout << "Formula:\n";
    cout << "mu_R(x,y) = min(mu_A(x), mu_B(y))\n\n";

    double R[3][2];

    for (int i = 0; i < 3; i++)
    {
       for (int j = 0; j < 2; j++)
       {
          R[i][j] = min(A[i], B[j]);

          cout << "mu_R("
               << X[i] << ", "
               << Y[j] << ") = min("
               << formatNumber(A[i]) << ", "
               << formatNumber(B[j]) << ") = "
               << formatNumber(R[i][j])
               << endl;
       }
    }

    cout << "\nCartesian Product R = A x B:\n\n";

    cout << setw(8) << " ";

    for (const string& y : Y)
```

```
        cout << setw(8) << y;

    cout << "\n";

    for (int i = 0; i < 3; i++)
    {
        cout << setw(8) << X[i];

        for (int j = 0; j < 2; j++)
        {
            cout << setw(8)
                 << formatNumber(R[i][j]);
        }

        cout << "\n";
    }
}

// QUESTION 3
void question3()
{
    cout <<
"\n============================================================\n";
    cout << "QUESTION 3 - FUZZY RELATION COMPOSITION\n";
    cout <<
"============================================================\n";

    double R[2][2] =
    {
        {0.6, 0.3},
        {0.2, 0.9}
    };

    double S[2][3] =
    {
        {1.0, 0.5, 0.3},
        {0.8, 0.4, 0.7}
    };

    double T_maxMin[2][3];
```

```
double T_maxProduct[2][3];

cout << "\nRelation R:\n";
cout << "         y1      y2\n";
cout << "x1       0.6     0.3\n";
cout << "x2       0.2     0.9\n";

cout << "\nRelation S:\n";
cout << "         z1      z2      z3\n";
cout << "y1       1.0     0.5     0.3\n";
cout << "y2       0.8     0.4     0.7\n";

// PART (a) MAX-MIN COMPOSITION

cout << "\n-------------------------------------------------------------\n";
cout << "(a) MAX-MIN COMPOSITION\n";
cout << "-------------------------------------------------------------\n";

cout << "Formula:\n";
cout << "T(x,z) = max_y [ min(R(x,y), S(y,z)) ]\n\n";

for (int i = 0; i < 2; i++)
{
   for (int k = 0; k < 3; k++)
   {
      double value1 = min(R[i][0], S[0][k]);
      double value2 = min(R[i][1], S[1][k]);

      T_maxMin[i][k] = max(value1, value2);

      cout << "T(" << i + 1 << "," << k + 1 << ") = max("
          << "min(" << formatNumber(R[i][0]) << ", "
          << formatNumber(S[0][k]) << "), "
          << "min(" << formatNumber(R[i][1]) << ", "
          << formatNumber(S[1][k]) << "))"
          << " = "
          << formatNumber(T_maxMin[i][k])
          << endl;
   }
}
```

```
cout << "\nResult T (Max-Min):\n";
cout << "          z1      z2      z3\n";

cout << "x1       ";
for (int k = 0; k < 3; k++)
   cout << setw(8) << formatNumber(T_maxMin[0][k]);
cout << "\n";

cout << "x2       ";
for (int k = 0; k < 3; k++)
   cout << setw(8) << formatNumber(T_maxMin[1][k]);
cout << "\n";

// PART (b) MAX-PRODUCT COMPOSITION

cout << "\n------------------------------------------------------------\n";
cout << "(b) MAX-PRODUCT COMPOSITION\n";
cout << "------------------------------------------------------------\n";

cout << "Formula:\n";
cout << "T(x,z) = max_y [ R(x,y) * S(y,z) ]\n\n";

for (int i = 0; i < 2; i++)
{
   for (int k = 0; k < 3; k++)
   {
      double value1 = R[i][0] * S[0][k];
      double value2 = R[i][1] * S[1][k];

      T_maxProduct[i][k] = max(value1, value2);

      cout << "T(" << i + 1 << "," << k + 1 << ") = max("
          << formatNumber(value1) << ", "
          << formatNumber(value2) << ") = "
          << formatNumber(T_maxProduct[i][k])
          << endl;
   }
}
```

```
    cout << "\nResult T (Max-Product):\n";
    cout << "          z1      z2      z3\n";

    cout << "x1        ";
    for (int k = 0; k < 3; k++)
        cout << setw(8) << formatNumber(T_maxProduct[0][k]);
    cout << "\n";

    cout << "x2        ";
    for (int k = 0; k < 3; k++)
        cout << setw(8) << formatNumber(T_maxProduct[1][k]);
    cout << "\n";
}
void interactiveFuzzyOperations()
{
    vector<int> elements = {0, 1, 2, 3, 4};

    char choice;

    do
    {
        cout <<
"\n============================================================\n";
        cout << "INTERACTIVE FUZZY SET OPERATIONS\n";
        cout <<
"============================================================\n";

        vector<double> A(5);
        vector<double> B(5);

        cout << "\nElements: 0, 1, 2, 3, 4\n";

        cout << "\nEnter membership values for A:\n";

        for (int i = 0; i < 5; i++)
        {
            do
            {
                cout << "A[" << elements[i] << "] = ";
                cin >> A[i];
```

```
        if (A[i] < 0 || A[i] > 1)
        {
            cout << "Enter a value between 0 and 1.\n";
        }

    } while (A[i] < 0 || A[i] > 1);
}

cout << "\nEnter membership values for B:\n";

for (int i = 0; i < 5; i++)
{
    do
    {
        cout << "B[" << elements[i] << "] = ";
        cin >> B[i];

        if (B[i] < 0 || B[i] > 1)
        {
            cout << "Enter a value between 0 and 1.\n";
        }

    } while (B[i] < 0 || B[i] > 1);
}

cout << "\n------------------------------------------------------------\n";
cout << "INPUT FUZZY SETS\n";
cout << "------------------------------------------------------------\n";

cout << "A = ";
printFuzzySet(A, elements);
cout << "\n";

cout << "B = ";
printFuzzySet(B, elements);
cout << "\n";

vector<double> A_union_B = fuzzyUnion(A, B);
vector<double> A_inter_B = fuzzyIntersection(A, B);
```

```
vector<double> A_bar = fuzzyComplement(A);
vector<double> B_bar = fuzzyComplement(B);

cout << "\n------------------------------------------------------------\n";
cout << "ALL FUZZY SET OPERATIONS\n";
cout << "------------------------------------------------------------\n";

printResult("(a) A U B",
        A_union_B,
        elements);

printResult("(b) A ∩ B",
        A_inter_B,
        elements);

printResult("(c) A-bar",
        A_bar,
        elements);

printResult("(d) B-bar",
        B_bar,
        elements);

printResult("(e) A U A-bar",
        fuzzyUnion(A, A_bar),
        elements);

printResult("(f) A ∩ A-bar",
        fuzzyIntersection(A, A_bar),
        elements);

printResult("(g) B U B-bar",
        fuzzyUnion(B, B_bar),
        elements);

printResult("(h) B ∩ B-bar",
        fuzzyIntersection(B, B_bar),
        elements);
```

```
        printResult("(i) A ∩ B-bar",
                fuzzyIntersection(A, B_bar),
                elements);

        printResult("(j) A U B-bar",
                fuzzyUnion(A, B_bar),
                elements);

        printResult("(k) B ∩ A-bar",
                fuzzyIntersection(B, A_bar),
                elements);

        printResult("(l) B U A-bar",
                fuzzyUnion(B, A_bar),
                elements);

        vector<double> m_result =
           fuzzyComplement(A_union_B);

        printResult("(m) (A U B)-bar",
                m_result,
                elements);

        vector<double> n_result =
           fuzzyIntersection(A_bar, B_bar);

        printResult("(n) A-bar ∩ B-bar",
                n_result,
                elements);

        cout << "\n------------------------------------------------------------\n";
        cout << "Do you want to enter another pair of fuzzy sets? (Y/N): ";

        cin >> choice;
        choice = toupper(choice);

    } while (choice == 'Y');

    cout << "\nProgram finished.\n";
}
```

```
int main()
{
    cout << "************************************************************\n";
    cout << "            FUZZY LOGIC OPERATIONS\n";
    cout << "************************************************************\n";

    question7();
    question9();
    question2();
    question3();

    interactiveFuzzyOperations();

    return 0;
}
```

Output:

```
tudents@students-HP-280-G3-SFF-Business-PC:~/Desktop/2024300170_Vedant$ g++ fuzzy.cpp -o fuzzy
tudents@students-HP-280-G3-SFF-Business-PC:~/Desktop/2024300170_Vedant$  ./fuzzy
**********************************************************
            FUZZY LOGIC OPERATIONS
**********************************************************

==========================================================
UESTION 7 - FUZZY SET OPERATIONS
==========================================================

iven:
 = { 0.1/0 + 0.2/1 + 0.4/2 + 0.6/3 + 1/4 }
 = { 1/0 + 0.5/1 + 0.7/2 + 0.3/3 + 0/4 }

a) A U B = { 1/0 + 0.5/1 + 0.7/2 + 0.6/3 + 1/4 }
b) A ∩ B = { 0.1/0 + 0.2/1 + 0.4/2 + 0.3/3 + 0/4 }
c) A-bar = { 0.9/0 + 0.8/1 + 0.6/2 + 0.4/3 + 0/4 }
d) B-bar = { 0/0 + 0.5/1 + 0.3/2 + 0.7/3 + 1/4 }
e) A U A-bar = { 0.9/0 + 0.8/1 + 0.6/2 + 0.6/3 + 1/4 }
f) A ∩ A-bar = { 0.1/0 + 0.2/1 + 0.4/2 + 0.4/3 + 0/4 }
g) B U B-bar = { 1/0 + 0.5/1 + 0.7/2 + 0.7/3 + 1/4 }
h) B ∩ B-bar = { 0/0 + 0.5/1 + 0.3/2 + 0.3/3 + 0/4 }
i) A ∩ B-bar = { 0/0 + 0.2/1 + 0.3/2 + 0.6/3 + 1/4 }
j) A U B-bar = { 0.1/0 + 0.5/1 + 0.4/2 + 0.7/3 + 1/4 }
k) B ∩ A-bar = { 0.9/0 + 0.5/1 + 0.6/2 + 0.3/3 + 0/4 }
l) B U A-bar = { 1/0 + 0.8/1 + 0.7/2 + 0.4/3 + 0/4 }
m) (A U B)-bar = { 0/0 + 0.5/1 + 0.3/2 + 0.4/3 + 0/4 }
n) A-bar ∩ B-bar = { 0/0 + 0.5/1 + 0.3/2 + 0.4/3 + 0/4 }
```

```
============================================================
QUESTION 9 - ALGEBRAIC AND BOUNDED OPERATIONS
============================================================

Given:
A = { 0.2/1 + 0.3/2 + 0.4/3 + 0.5/4 }
B = { 0.1/1 + 0.2/2 + 0.2/3 + 1/4 }

(a) Algebraic Sum
    Formula: A + B - A*B
    Result = { 0.28/1 + 0.44/2 + 0.52/3 + 1/4 }

(b) Algebraic Product
    Formula: A*B
    Result = { 0.02/1 + 0.06/2 + 0.08/3 + 0.5/4 }

(c) Bounded Sum
    Formula: min(1, A+B)
    Result = { 0.3/1 + 0.5/2 + 0.6/3 + 1/4 }

(d) Bounded Difference
    Formula: max(0, A-B)
    Result = { 0.1/1 + 0.1/2 + 0.2/3 + 0/4 }
```

```
============================================================
QUESTION 2 - FUZZY CARTESIAN PRODUCT
============================================================

Given:
A = { 0.3/x1 + 0.7/x2 + 1/x3 }
B = { 0.4/y1 + 0.9/y2 }

Formula:
mu_R(x,y) = min(mu_A(x), mu_B(y))

mu_R(x1, y1) = min(0.3, 0.4) = 0.3
mu_R(x1, y2) = min(0.3, 0.9) = 0.3
mu_R(x2, y1) = min(0.7, 0.4) = 0.4
mu_R(x2, y2) = min(0.7, 0.9) = 0.7
mu_R(x3, y1) = min(1, 0.4) = 0.4
mu_R(x3, y2) = min(1, 0.9) = 0.9

Cartesian Product R = A x B:

              y1      y2
      x1     0.3     0.3
      x2     0.4     0.7
      x3     0.4     0.9
```

```
============================================================
QUESTION 3 - FUZZY RELATION COMPOSITION
============================================================

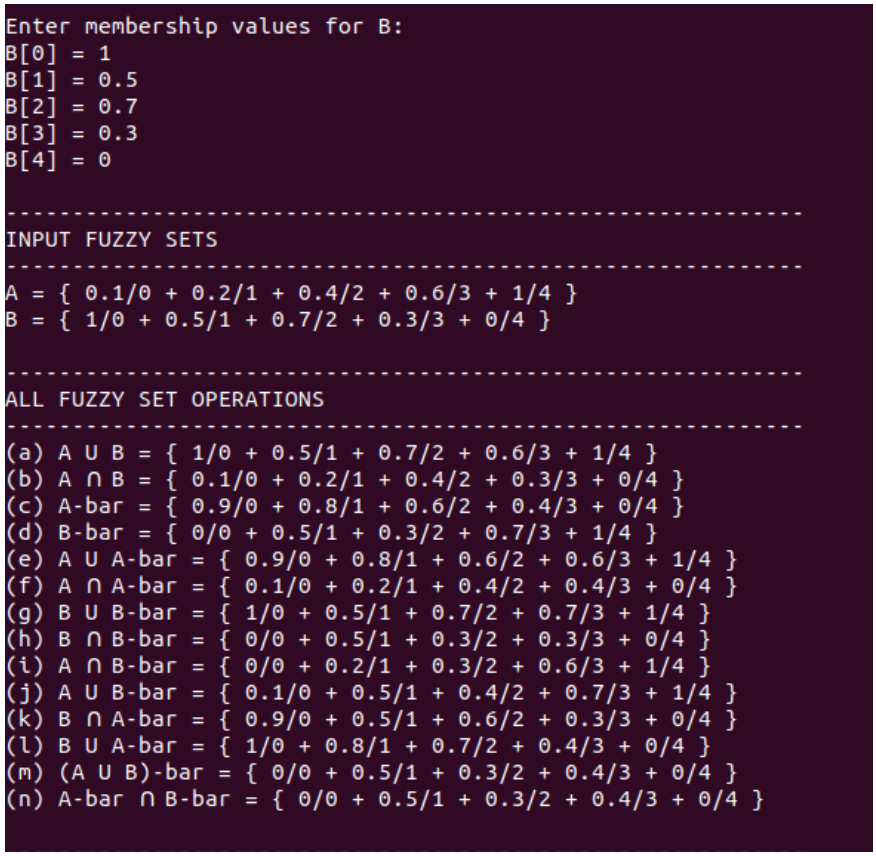
Relation R:
          y1      y2
x1       0.6     0.3
x2       0.2     0.9

Relation S:
          z1      z2      z3
y1       1.0     0.5     0.3
y2       0.8     0.4     0.7

------------------------------------------------------------
(a) MAX-MIN COMPOSITION
------------------------------------------------------------
Formula:
T(x,z) = max_y [ min(R(x,y), S(y,z)) ]

T(1,1) = max(min(0.6, 1), min(0.3, 0.8)) = 0.6
T(1,2) = max(min(0.6, 0.5), min(0.3, 0.4)) = 0.5
T(1,3) = max(min(0.6, 0.3), min(0.3, 0.7)) = 0.3
T(2,1) = max(min(0.2, 1), min(0.9, 0.8)) = 0.8
T(2,2) = max(min(0.2, 0.5), min(0.9, 0.4)) = 0.4
T(2,3) = max(min(0.2, 0.3), min(0.9, 0.7)) = 0.7

Result T (Max-Min):
          z1      z2      z3
x1            0.6     0.5     0.3
x2            0.8     0.4     0.7
```

	
CONCLUSION:	Thus, I learned that fuzzy logic helps machines make decisions in a more human-like way. It handles uncertain or approximate information using values between 0 and 1. It is useful in many real-life applications such as washing machines, air conditioners, and robotics.